

技术分享 · 2026

智能体 RAG

Agentic RAG

从检索流水线，到会思考的自主研究智能体



主线：RAG 的演进史 = 检索决策权从「人」不断让渡给「模型」

一条主线：检索「决策权」的迁移



一句话记住：决策权往右走一格，系统就更「智能」一分，但也更贵、更难控。后面每一节都挂在这条线上。

今天的路线图



1 · RAG 基础与技术演进

Naive → Advanced → Modular 三段论



3 · 什么是 Agentic RAG

定义、三条标准、四种架构范式



4.5 · Deep Research

Agentic RAG 的「完全体」



6 · 开源框架选型

LangChain / LlamaIndex / RAGFlow...



2 · 经典 RAG 方法

Self-RAG / CRAG / GraphRAG 家族



4 · 2026 前沿

RL 检索智能体、智能体记忆、范式之争



5 · 踩坑实战（干货核心）

痛点、卡点、根因与解法



7 · 评估与上线 checklist

RAGAS、可观测、治理

为什么 Naive RAG 不够用？

“ 「我们 Q3 销量最高的产品，它的主要竞品去年发布了什么新功能？」

步骤 1
查 Q3 销量数据



步骤 2
定位销量第一的产品



步骤 3
找出它的主要竞品



步骤 4
查竞品去年的新功能



单次「检索一次→生成」注定答不好。

这是一条 多步、跨源、有依赖 的链路：第 3 步要等第 2 步的结果，第 4 步要等第 3 步。Naive RAG 没有「判断要不要再查、该查什么」的能力——这正是 Agentic RAG 要解决的核心问题。

01

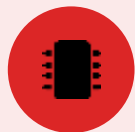


PART 1

RAG 基础与技术演进

从原始 RAG 到「Naive → Advanced → Modular」经典三段论

RAG 在解决什么问题



参数化记忆

模型权重里的知识

静态、会过时；训练截止后的事一无所知；记不住私有数据；答错也「很自信」。



非参数化记忆

外部可检索的语料

实时、可更新、可溯源；改文档即改答案；能引用来源；隔离私有知识库。

RAG = 把两者结合：



一句话： RAG 给一个「静态的大脑」接上了一块「可随时更新、可查证出处」的外部硬盘。

经典三段论：Naive → Advanced → Modular



Naive RAG

索引 → 检索 → 生成 (一条直线)

典型痛点

召回噪声大、chunk 切碎语义、查不到就硬答（幻觉）、无法多跳



Advanced RAG

在直线上加「前处理+后处理」

带来什么

查询改写 / HyDE / 混合检索 / 重排 / 上下文压缩



Modular RAG

拆成可替换、可编排的模块

带来什么

检索 / 记忆 / 路由 / 融合自由组合——离 Agent 只差「谁来编排」

讲法提示：把「前-中-后+模块化」当成一棵树，后面所有方法都能挂上去。

① 查询改写 Query Rewriting



是什么 检索前，把用户的原始问题改写成「检索器更容易命中」的形式——因为用户怎么问、和文档怎么写，往往对不上。

四种常见改写

口语化改写：把模糊口语变成可检索的关键词式查询

子问题分解：复杂问题拆成多个可独立检索的子查询 (multi-query)

退一步提问：step-back：先问更上位的概念，再回到细节

指代消解：多轮对话里把「它 / 这个」还原成具体实体

示例：多轮对话中的指代

原始（上下文缺失）

上一轮：我们在看 ProductX
本轮："它最大支持多少并发?"
→ 直接拿 "它" 检索 = 检索器懵

改写后

指代消解 + 补全：
"ProductX 最大并发连接数 规格"
→ 精确命中规格文档

多轮对话里，指代与省略是检索失败的高发区



干货 / 坑：多轮场景下，先做查询改写常比换更大的 embedding 模型更划算。坑：改写可能「脑补」出文档里没有的实体——约束模型只用已知上下文改写，并保留原查询做兜底。

② HyDE 假设性文档嵌入



是什么 先让 LLM 根据问题「编」一段假设答案，再用这段假设答案的向量去检索——而不是用问题本身的向量。

为什么有用

错位问题：问题短、答案长，措辞还不同，二者在向量空间天然不对齐

核心技巧：「假答案」和真实文档长得更像 → 检索更准

零样本：不需要标注数据，冷启动 / 缺训练数据时尤其香

适用：专业领域、问法与原文差异大的场景

示例：RAG 为什么会幻觉？

普通检索 (embed 问题)

```
embed("RAG 为什么会幻觉?")  
→ 召回一堆「RAG 是什么」的  
泛泛介绍段落
```

HyDE (embed 假设答案)

```
LLM 先写："幻觉源于检索失败、  
上下文不足、信息冲突..."  
embed(这段) → 命中真正的成因段落
```

用「答案的样子」去找答案，比用「问题的样子」更准



干货 / 坑：HyDE 在冷启动和垂域问答上提升明显。坑：模型不懂的领域会把假答案编歪反而带偏检索；且多一次 LLM 调用加延迟——高并发时建议只在「首轮召回置信度低」时才触发。

③ 混合检索 Hybrid Retrieval



是什么 同时跑稠密向量检索（懂语义）和稀疏检索（BM25 / 关键词，懂字面），再把两路结果融合——取长补短。

要点

向量擅长：同义、近义、概念相关（「响应慢」≈「性能优化」）

BM25 擅长：精确字面：型号、错误码、人名、缩写、专有名词

融合用 RRF：按排名倒数合并，不必对齐两路的分数尺度

中文要点：BM25 依赖分词，分词不好 = BM25 白上

示例：查「错误码 ERR_2048」

纯向量

把 ERR_2048 当普通文本
→ 召回泛泛的「错误处理」段落
漏掉那条精确文档

向量 + BM25 (RRF 融合)

BM25 精确命中含 ERR_2048 的那条
向量补充语义近邻
→ 既准又全

多数复盘里，混合检索带来 20–30% 量级的准确率提升



干货 / 坑：混合检索是 2026 生产基线，几乎应默认开启。坑：两路分数不可直接相加（尺度不同）——用 RRF 或先归一化；中文务必配好分词器。

④ 重排 Reranking



是什么 两阶段检索：先用快的检索器粗召回 top-N（如 50），再用慢但准的 cross-encoder 精排，只留 top-k（如 5）喂给 LLM。

为什么要两段

双塔 bi-encoder：query 与 doc 分开编码，快、可预存，但牺牲细粒度交互

交叉编码 cross-encoder：query+doc 拼一起过模型，精度高但贵，只能用于小候选集

先粗后精：向量负责广撒网，reranker 负责挑出真正相关的

性价比之王：常是改动最小、收益最大的一招

示例：排序纠正

仅向量召回

#1 语义沾边但跑题
#2 #3 相关度参差
真正的答案被埋在 #7

cross-encoder 重排后

#1 ← 原来的 #7（真答案）
精排理解了 query-doc 细粒度相关
只留 top-5，聚焦且省 token

召回负责「找得到」，重排负责「排得对」



干货 / 坑：重排往往是性价比最高的优化。坑：reranker 也有延迟 / 成本，候选 N 越大越贵——控制 N；上游切分烂，reranker 也救不回来；领域差异大时考虑微调或换领域 reranker。

⑤ 上下文压缩 Context Compression



是什么 召回之后、喂给 LLM 之前，把上下文里和问题无关的内容删掉或压缩，只留真正相关的部分。

两种路线 + 动机

抽取式：从 chunk 里挑出相关句子（快、不增加 LLM 调用）

摘要 / 压缩式：让 LLM 把上下文压成精简版（更狠，但多一次调用）

prompt 压缩：如 LLMingua，按信息量删冗余 token

动机：降 token 成本 + 缓解「lost in the middle」

示例：压缩前后

压缩前

召回 5 段 $\approx$ 4000 token
真正相关的其实只有 3 句话
噪声多、贵、还可能被淹没

压缩后

抽取 / 摘要 $\rightarrow \approx$ 600 token
只留命中问题的句子
更准、更快、更便宜

召回是「多多益善」，喂给 LLM 却要「少而精」



干货 / 坑：压缩能降本并缓解 lost-in-the-middle。坑：抽取式遇到「分散在多句的事实」可能漏关键信息；摘要式又多一次 LLM 调用和延迟——对延迟敏感选抽取式，对质量敏感选摘要式但要缓存。

02



PART 2

经典 RAG 方法

会「反思」的 RAG、以及补上关系推理的 GraphRAG 家族

会「反思」的 RAG：通往 Agentic 的过渡形态

Self-RAG

训练模型生成「反思 token」，自己决定何时检索，并对结果与自己的输出做自我批判

CRAG

加一个轻量评估器给检索结果打分；质量差就触发纠错（改写查询 / 退回网页搜索）

FLARE

生成过程中，当模型对下一句不确定时主动触发检索，而非开头一次查完

RAPTOR

对语料递归聚类+摘要，建「多粒度摘要树」，可在不同抽象层级命中——擅长全局概览

共同点：它们开始把「要不要再查、查得好不好」交给系统判断——这就是「半智能体」过渡带。

GraphRAG 家族：补上「关系推理 / 多跳」

向量检索擅长「语义相似」，但不擅长「关系推理」。知识图谱补上这块。

方法	核心思路	特点 / 适用
Microsoft GraphRAG	LLM 抽实体-关系建图，Leiden 聚类生成「社区摘要」，分 local/global 查询	擅长全局、汇总性问题；建图成本高
LightRAG	双层检索（实体 + 主题），图结构 + 向量结合	更轻、更快、增量更新友好
HippoRAG / HippoRAG 2	仿海马体记忆，在图上跑 Personalized PageRank 做多跳	多跳推理便宜、可持续学习
LazyGraphRAG	不预先建全图，查询时按需构图	大幅降低索引成本



诚实结论：没有一种图谱 RAG 通吃所有场景。图谱 RAG 的价值随「查询复杂度」上升而上升——简单 QA 用它反而浪费算力。

什么时候该上图谱 RAG? 一个例子

问题：「与图灵奖得主 x 合作过、且现在在创业公司任 CTO 的人，他们公司做什么？」



纯向量检索

按「语义相似」捞段落，但这题要的是「合作关系→职位→公司」三跳串联。

向量库里没有任何一段同时讲全这条链 → 召回一堆「沾边但不对」的段落，答案流畅却错。



GraphRAG

图上存的就是「x —合作— 某人 —任职— 公司」这种关系边。
沿边多跳遍历，精确锁定符合三个条件的人和公司 → 答案可追溯、可解释。

选型口诀：问题里有「关系、多跳、谁和谁」→ 考虑图谱；只是「找相似内容」→ 向量/混合就够。

03



PART 3

什么是 Agentic RAG

把会推理的 Agent 嵌入检索流程——决策权交给模型

核心转变：从「查完就答」到「边想边查」

Naive RAG (静态)

query → 检索一次 (top-k)

→ 拼上下文

→ LLM 生成

→ 结束 (不管够不够)

问题：检索错了 / 不全，也只能硬着头皮答。不会判断、不会重试。

Agentic RAG (动态)

思考：要不要检索？查什么？

→ 检索 → 观察结果

思考：够了吗？缺什么？

→ (不够) 换个查询再检索

→ (够了) 综合 → 作答

↑ 循环，直到自认为足够

关键：系统先「推理」再行动，决定要不要查、查什么、够没够。

何为「真正的 agentic」：三条标准

用这把尺回看前面：Self-RAG / CRAG 只满足一部分， workflow 仍受预设约束——所以是「过渡形态」，不是完全体。

1

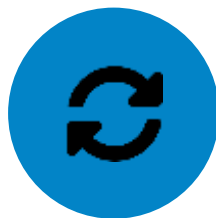


自主策略

Autonomous Strategy

由模型决定检索策略，而不是套用预设算法

2



迭代执行

Iterative Execution

能多轮循环，根据中间结果调整下一步

3



交错工具使用

Interleaved Tool Use

推理与工具调用交织进行，而非先查完再想

来源：A-RAG (arXiv 2602.03442) 提出的三原则

四种主流架构范式（由简到繁）



1·路由型

一个 Agent 选走哪条路：向量库 / 图谱 / 网页 / SQL。最轻量



2·规划型 (ReAct)

拆子问题，思考→检索→观察→再思考，直到够了。2026 最主流



3·多智能体

编排 Agent 调度检索/分析/汇报子 Agent，各自可独立优化



4·图上多智能体

在异构知识图谱上多智能体自适应推理（前沿研究形态）

规划型 (ReAct) 的真实轨迹长这样：

Thought: 要回答竞品功能，先得知道 Q3 销量第一的产品是什么

Action: search("Q3 销量 排行") → Observation: 第一名是 ProductX

Thought: 现在找 ProductX 的主要竞品

Action: search("ProductX 竞品") → Observation: 主要竞品是 ProductY

Thought: 信息够了，查 ProductY 去年新功能即可作答 → Final Answer ...

架构选型决策树（实用速查）



核心原则：从简单开始，复杂度只在需要时引入——每加一层都加延迟、成本和故障面。

04



PART 4

2026 前沿

RL 把「会搜索」训进权重、智能体记忆、以及 RAG 的范式之争

把「会搜索」训进模型：Search-R1

最重要的范式转移：不再靠提示词「教」模型怎么搜，而是用强化学习「训」出来。

推理与检索「交错」的训练轨迹：

```
<think> 我不确定 X, 需要查一下 </think>

<search> X 的最新数据 </search>

<information> ...检索回来的证据... </information>

<think> 还差一个条件, 再查 </think>

<search> ... </search> → <information> ... </information>

<answer> 综合得出结论 </answer>
```



结果奖励

答对就给分 (outcome-based), 不需要标注中间步骤



检索 token 掩码

对检索回来的 token 做 masking, 稳定 RL 训练



训练方法

GRPO / PPO / REINFORCE, 受 DeepSeek-R1 启发

+41%

在 7 个 QA 数据集上, Search-R1 (Qwen2.5-7B) 相比各类 RAG 基线提升约 41%。

意义: 检索从「外挂的工程 pipeline」变成了模型的内生能力——回到主线, 决策权这次直接进了模型权重。

同一谱系 + 把检索接口交给模型

RL 检索智能体谱系

- **Search-o1** — 在文档中推理 (Reason-in-Documents)
- **R1-Searcher** — 用 RL 激发自主搜索能力
- **DeepRAG** — 逐步思考、按需检索
- **RAG-R1** — 多查询并行, 降低推理时延
- **R3-RAG / SSRL** — 学习检索-推理交错 / 自搜索



A-RAG: 分层检索接口

不再写死检索算法, 而是把三种接口直接暴露给模型:

关键词搜索

语义搜索

读取 chunk

让模型像人查资料一样, 自己决定下一步用哪种。

并随「模型规模 + 测试时算力」扩展——想得越久、查得越多, 答得越好。

RAG ≠ 记忆：2026 的一个重要分野

RAG = 无状态检索

每次从外部索引取 chunk，会话之间清零。回答「语料里写了什么」。

智能体记忆 = 有状态

跨会话存储并演化上下文。回答「这个 Agent 学到了什么、事实有没有变」。

三层记忆架构：



情景记忆

episodic — 交互历史



语义记忆

semantic — 事实与实体关系



程序记忆

procedural — 学到的任务模式



「记忆幻觉」示例：

Agent 记忆里既有旧事实「公司 CEO 是张三」，又有新事实「CEO 是李四」——若把两者混在一起，会合成出一个自信的错误答案。把 RAG 和 Memory 混为一谈，正是这种 bug 的根源。

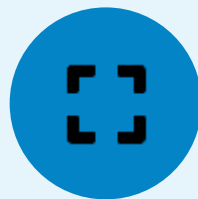
范式之争：RAG 会消失吗？



「RAG 时代要结束了」

有厂商（如 Pinecone Nexus + KnowQL）主张构建「为 Agent 而非为人」的知识层，宣称某金融任务把 280 万 token 压到 4 千（约 98%↓）。

注：厂商内部基准，尚未客户验证



「长上下文会取代 RAG」

2026 前沿模型上下文达百万 token 级，改变成本结构。但企业普遍认为：超长上下文成本高、治理风险大。

RAG 在精准/权限/成本上仍不可替代



「务实共存」（主流）

2026 企业基线是 Hybrid RAG（向量+关键词）；只在推理深度确需时才上 Graph / Agentic RAG。

RAG 已是一「族」模式，按复杂度选型

现场互动：举手投票（会消失/长上下文取代/务实共存），再揭示三派观点。

Deep Research: Agentic RAG 的「完全体」



普通 RAG 「一个问题查一次」；Deep Research 像一个尽职的分析师：

拆子问题 → 跨几十上百个源广搜 → 读全文而非摘要 → 交叉验证 → 找出还缺什么 → 补搜 → 综合成带引用的多页报告。

一句话定位

Deep Research = Agentic RAG 把「规划 + 迭代检索 + 反思 + 多智能体」全部拉满的长程版本。本场前面讲的所有要素，它都用上了。

怎么来的

2024 年底由 OpenAI / Google / Perplexity / xAI 掀起，2025–2026 全面普及。如今 ChatGPT、Gemini、Claude 都已内置。



为什么放这里讲：它是把整场所有概念落到一个大家都用过的真实产品形态上的最佳收口案例——理论到此全部「显形」。

拆开看：Deep Research 的架构循环



步骤3-5 循环往复，这是和「查一次」最本质的区别

多智能体变体：一个 research supervisor 把查询切成子主题，并行 spawn 多个 sub-agent 各自搜读带引用返回，supervisor 判断够没够再编排成稿——正是「多智能体编排」范式的真实落地。

学术 ↔ 产品 两端，以及怎么落地

Search-R1 (学术)



Deep Research (产品)

OpenAI Deep Research 本身就是一个用 RL 训练的推理模型 (o3 特化) ——和 Search-R1 是同一条技术路线的两端。



能力已 API 化

Gemini 已通过 Interactions API 开放（异步、后台执行、轮询取结果），可直接接进自己的工作流。



开源可自搭

LangChain Open Deep Research (supervisor + 并行 sub-agent + 可插 MCP)、qx-labs 迭代研究器、Cognitive Kernel-Pro。



务实提醒

很强但慢且贵（一次跑几分钟、大量 token）。给「低频、高价值、要成体系结论」的任务用，别替代高频 QA。

量级参照：在 GAIA、Humanity's Last Exam 等高难基准上，Deep Research 类系统显著优于普通模型（如 OpenAI 在 GAIA 约 67% pass@1）。各家口径不一，引用时注明来源。

05



PART 5 · 干货核心

踩坑实战：RAG 落地的痛点与解法

不讲空话，讲真实失败长什么样、根因在哪、怎么修

≈73%

生产 RAG 故障出在「检索」层，而非生成

≈80%

检索故障的根因在「入库 + 切分」，不在 LLM

数据为从业者复盘量级直觉；经典文献：
Seven Failure Points (arXiv 2401.05856)

坑 1 · 检索失败是「静默」的（最危险）



真实案例：

某合规知识库上线三周，有人问「承包商入职流程」。系统答得自信、结构完整，却漏掉了「受监管项目承包商」的例外条款——这条款其实在文档里、也被正确嵌入了，是检索没把它召回。在合规场景，这种「差一点」就是事故。



症状

检索失败不抛异常。它返回一个流畅、像模像样、但在关键处错了的答案。Demo 看不出，真实流量下变成「信任的慢性流失」。



根因

该召回的关键段落没被召回——可能是切分切散了、可能是查询和文档措辞对不上、可能是 top-k 太小被挤掉。



解法

① 第一天就上评估埋点（RAGAS），否则只能靠直觉 debug；② 调试顺序：先查「到底什么 context 进了 LLM」，再碰 prompt。检索质量决定答案天花板。

一句话：先修生成是错的——检索没召回，再好的 prompt 也救不回来。

坑 2 · 切分：上游崩了，下游再优化也白搭

固定 512-token 切分（反面）

...承包商需在入职前完成 A、B。

—— 切  chunk 边界 ——

例外：受监管项目承包商还需 C。

主条款和例外条款被切到两个 chunk → 检索只命中前半，例外永远召回不回。

结构感知切分（正面）

【承包商入职 · 完整小节】

入职前完成 A、B；

例外：受监管项目还需 C。 ← 同一 chunk

按语义/结构边界切，主条款与例外留在一起 → 一次召回就完整。

解法（代价从低到高）：

1. 保留文档结构：先用好解析器保住表格/标题/阅读顺序，别一上来 flatten 成纯文本
2. 语义 / 递归切分：在自然边界（段落、小节、主题切换）切，而非按固定 token 硬切
3. 给每个 chunk 挂元数据：来源、章节、生效日期、权限——很多「召回不准」其实是缺元数据没法过滤
4. 上下文增强切分（contextual / late chunking）：给 chunk 补「它在全文里的位置」再嵌入
5. 能不切就不切： workflow 没强需求时，保持文档完整

坑 3 · 纯向量不够：混合检索 + 重排

症状

知识库一变大，向量检索就开始返回「语义相似但其实不相关」的 chunk，答案流畅却事实错误；而且纯向量会漏掉明显的关键词精确匹配。

示例：用户查「错误码 ERR_2048」

纯向量：把 ERR_2048 当普通文本，召回一堆「讲错误处理」的泛泛段落，漏掉那一条精确文档。 **+BM25：**关键词精确命中含「ERR_2048」的那一段。两者并用，互补。



混合检索

向量（语义）+ BM25（关键词）并用，多数复盘里准确率提升 20–30% 量级。



重排 Rerank

原始召回常有 20–50 个候选，全塞给 LLM 又贵又稀释。cross-encoder 打分→只留最相关 5 个，精度与聚焦度大涨。

坑 4 · 延迟成本 坑 5 · 嵌入漂移

坑 4 · 延迟与成本

症状：加大 top_k、加 reranker、加长上下文、加多轮检索——召回上去了，但系统慢到用户等不起，token 成本也线性上涨（一次问答常要多次 LLM 调用）。

解法：

- 把「准确率 vs 延迟/成本」当显式取舍来设计
- 分级模型：简单步用小模型、关键步用大模型
- 缓存 + 限制 Agentic 循环的工具调用深度
- 设 token/成本预算、超时与熔断，监控 P95 延迟与单次成本

坑 5 · 嵌入漂移与陈旧

症状：上线那天最好，之后慢慢烂。换了 embedding 模型、文档集变了、用户措辞变了，旧索引慢慢失准；或系统拿六周前的旧版文档作答（「上周改了，它还在用旧的」）。

解法：

- 源一更新就重建索引（webhook 触发）
- 在 prompt 里盖「生效日期 / 版本」
- 缓存按内容哈希自动过期
- 把 embedding 版本、文档集版本纳入监控，定期重嵌 + 回归测试

坑 6 · Agentic 特有的坑



推理循环失控

Agent 反复查同一类东西打转

→ 硬上限（最多几轮）+ 预算 + 熔断 + 超时



权限串味

多租户索引里「数据bleed」

→ 检索层做 security trimming, 把用户角色传给 retriever, 多租户隔离要测



提示注入 / 工具滥用

检索回来的内容里藏指令

→ 工具白名单 + schema 校验 + 输出过滤 + 检索沙箱（参考 OWASP LLM）



本段总纲：先把数据管线做对，再谈花哨的检索策略。80% 的 RAG 问题，在你写第一行检索代码之前就已经决定了。

主流开源框架与选型

框架	定位	最适合	注意点
LlamaIndex	数据接入 + 检索	大语料精准检索、复杂 ingestion	Agent 能力不如 LangChain 体系
LangChain / LangGraph	编排 + Agent	检索只是众多能力之一的 Agent 系统	抽象多、版本变动快
Haystack	企业生产	强合规、要审计、要稳定	学习曲线陡、社区较小
RAGFlow	可视化文档 RAG	非开发者、文档解析重	资源占用重、LLM 支持面窄
Dify	低代码平台	快速搭原型 / 内部工具	灵活性不如 code-first
DSPy	程序化优化	自动调优 pipeline 而非手调 prompt	心智模型新、需适应
LightRAG / Pathway	图谱 / 流式	多跳低资源 / 数据持续变化	功能聚焦、非全栈



最实用的一句：别在 LlamaIndex 和 LangChain 间二选一——生产团队普遍组合用：LlamaIndex 做接入与检索 + LangGraph 做编排与 Agent。框架不是问题，喂进去的数据才是。

评估：为什么 Agentic RAG 不一样



普通 RAG 「一问一答」打分： Agentic RAG 是「多步轨迹」——终答对 ≠ 过程对（可能瞎蒙对、绕远路、超预算）；终答错了，也得知道是哪一步崩的。所以评估要分层。



① 逐查询指标

per-query

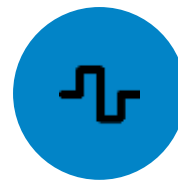
RAGAS 等：单次问答的检索质量 + 生成质量（忠实度、相关性）



② 轨迹追踪

trajectory

Phoenix / Langfuse / LangSmith：摊开 Agent 每一步，debug 规划错在哪



③ 漂移监控

drift

对一个「冻结的黄金集」每周回归，盯知识库 / 嵌入 / 评估漂移



Insight： 评估的第一价值是「定位故障层」（回扣 Part 5：73% 故障在检索）。三层缺任意一层，系统都会在 dev 里看着没事、上了生产却静默退化。三层是底线，不是可选项。

组件层指标：用 RAGAS 做「诊断」

两层指标

检索层 Recall@K · Precision@K · MRR · nDCG · Context Precision / Context Recall

生成层 Faithfulness (忠实度 / 反幻觉) · Answer Relevance (答没答到点) · Answer Correctness

生产参考阈值 (示例, 口径不一) :

faithfulness ≥ 0.9 · answer relevancy ≥ 0.85 · context precision ≥ 0.8

诊断技巧：交叉看指标定位根因

context recall 低 → 检索没召回 (上游问题, 先修检索)

recall 高但 faithfulness 低 → 召回对了但模型没用好 / 幻觉

relevance 低但 faithfulness 高 → 忠实于检索内容, 却没回答到问题

一个反直觉事实:

研究发现 47-67% 的查询里, 生成器忽略了检索器排第一的文档; 检索准确率只解释约 60% 的质量方差——剩下 40% 在「模型有没有用好上下文」。

Insight: 别只盯一个总分。Faithfulness 和 Answer Relevance 必须分开看——把「检索没召回」「召回了没用好」「答非所问」三类故障区分开, 才知道该去修检索、修 prompt、还是修查询理解。

轨迹层：Agentic 特有的评估（核心）

终答之外，更要评 Agent 「怎么做」的——这是和普通 RAG 评估最不同的一层。



工具调用正确性

选对工具了吗？参数对吗？



检索决策

该查时查了吗？不该查时停了吗？



端到端任务成功

research agent：忠实 ≠ 行动正确



步骤效率

用了几步、有无冗余检索、是否陷入循环



推理 / 反思质量

每一步的 thought 是否合理



成本 & 延迟

每任务 token、工具调用次数、P95

方法： LLM-as-judge + rubric · 与参考轨迹比对（trajectory matching） · 过程奖励 / process reward



Insight： 分清 outcome eval（答案对不对）和 process eval（过程好不好）。只看答案 → 放过「对的答案 + 失控的成本」；只看过程 → 容易过度优化。生产环境两个都要。

垂域 / 特定任务，到底怎么评？



通用 benchmark (GAIA、HotpotQA) 测不出你的垂域能力。 公开榜单高分 ≠ 在你的生产数据上好——必须自建「黄金评估集」。

黄金集怎么建 (4 步)

- 1 **真实分布**：从生产 query 日志 + 专家工单采样，别用想象的问题或噪声合成
- 2 **专家标注**：领域专家给参考答案，并标注「哪些源文档是必需的」
- 3 **刻意纳入硬样本**：多跳、需例外条款、易混淆术语、跨文档、表格抽取
- 4 **纳入「应拒答」样本**：没有答案的问题也要测——考幻觉抵抗

任务特定 rubric (不是一刀切相似度)

合规问答 faithfulness 权重最高；漏关键条款 = 直接判负

客服 可执行性 / 解决率 / 转人工率

医疗·金融 引用可溯源 + 保守性 (不确定就说不知道)

对齐业务 KPI：把「答案质量」映射到解决率、转人工率、合规事故数，而非只看学术分。



Insight：黄金集的质量 = 评估的天花板。50 条领域专家精标的样本，胜过 5000 条 LLM 自动生成的。先小而准，再扩；持续把生产里的失败「收编」成新测试用例。

流程、judge 的坑、工具与上线门禁

流程 + judge 的坑

离线 对黄金集跑回归，每次改动都跑；CI 门禁——关键指标低于阈值就 block 发布。

在线 线上采样 + 用户反馈 + A/B；按检索路由的滚动均值精度当漂移信号。

评估器悖论 (judge 的坑)

让会幻觉的同一个模型评检索 = 循环依赖；judge 还偏长答案、偏位置、偏自己。

解法：多 judge / 双模型集成 · 冻结黄金集只看趋势 · 人工抽查 5% trace · judge 要比被评模型更强。

上线前 checklist

- ☒ 数据管线：解析保结构、切分有策略、chunk 挂元数据
- ☒ 检索：混合检索 + 重排已上，top_k 实测调参
- ☒ 评估：三层评估埋点上线前就跑，有冻结回归集
- ☒ 成本 / 延迟：设预算、超时、熔断；监控 P95 与单次成本
- ☒ 安全：security trimming、工具白名单、注入防护、隔离已测
- ☒ 新鲜度：源更新触发重建、生效日期入 prompt、缓存过期
- ☒ 治理：trace 全量留存、审计日志、高风险步 human-in-loop

工具地图：RAGAS（探索）· DeepEval（CI/CD）· Phoenix / Langfuse / LangSmith（轨迹追踪）· Patronus / Galileo / TruLens（生产监控）



一句话：相信趋势，别迷信单个数字。（Gartner：到 2027 年超 40% agentic 项目被取消，多败在治理与成本，而非模型能力。）

四句话带走



1

RAG 已从一条流水线，长成一族模式

Naive → Advanced/Modular → Graph → Agentic → Deep Research 长程完全体



2

主线是检索决策权的迁移

人写死 → 模型运行时决策 → RL 训进权重；Search-R1 与 Deep Research 是同一路线两端



3

但 80% 的胜负在数据管线

解析、切分、元数据做对了再谈花哨检索；框架不是问题，喂进去的数据才是



4

工程上保持理性

按查询复杂度选型，从简单开始；上线前先建评估、先做治理

延伸阅读 & Q&A

核心论文

- Survey on Agentic RAG — arXiv 2501.09136
- RAG for LLMs: A Survey (三段论) — 2312.10997
- Search-R1 — 2503.09516 (RL 检索智能体)
- A-RAG 分层检索接口 — 2602.03442
- Self-RAG — 2310.11511 · GraphRAG / LightRAG / HippoRAG
- Seven Failure Points — 2401.05856 (踩坑必读)

框架·工具·产品

- 框架: LlamaIndex · LangChain/LangGraph · Haystack · RAGFlow · Dify
- 向量库: Milvus · Qdrant · Weaviate · pgvector
- 评估: RAGAS · LangSmith · Arize Phoenix · DeepEval
- Deep Research: Gemini Interactions API · LangChain Open Deep Research
- 持续跟踪: awesome-generative-ai-guide (RAG research table)



留给大家的问题:

- ① 我们的系统里, 第一个值得上 Agentic 能力的环节是什么? ② 哪类调研最值得交给 Deep Research?

谢谢!